

Experiment-13

To Implement Minimax Algorithm for Gaming

Aim

To implement the **Minimax Algorithm** for a two-player gaming problem using Python.

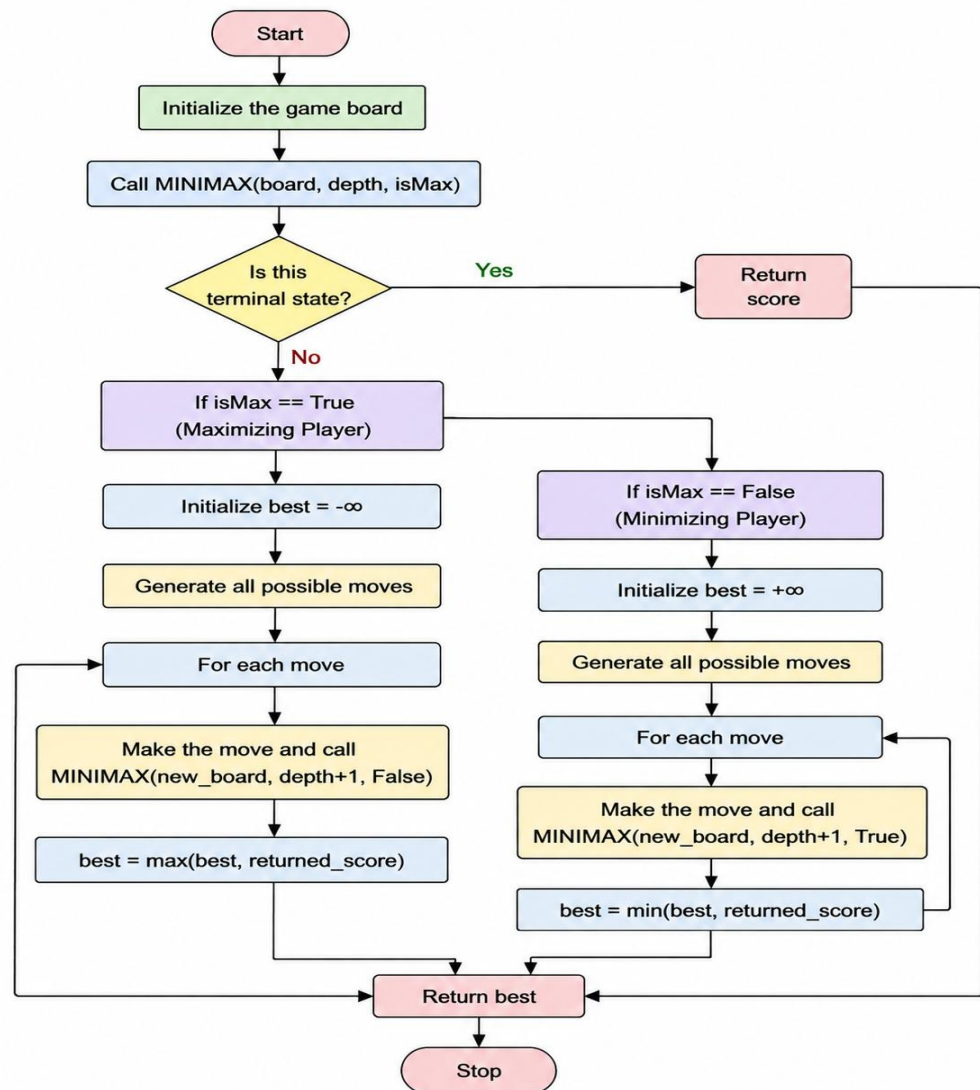
Objective

- To implement the Minimax Algorithm.
- To find the best possible move for a player.
- To understand decision-making in two-player games.

Algorithm

1. Start the program.
2. Initialize the game board.
3. Check whether the game is over.
4. If the maximizing player is playing, select the maximum score.
5. If the minimizing player is playing, select the minimum score.
6. Recursively evaluate all possible moves.
7. Select the best move based on the score.
8. Display the best move.
9. Stop the program.

Flowchart:



Code:

```
def minimax(board, depth, is_max):
```

```
    if 'O' in board:
```

```
        return 1
```

```
    if 'X' in board:
```

```
        return -1
```

```
    if ' ' not in board:
```

```
        return 0
```

```
    if is_max:
```

```

best = -100

for i in range(9):
    if board[i] == ' ':
        board[i] = 'O'
        best = max(best, minimax(board, depth + 1, False))
        board[i] = ' '

return best

else:
    best = 100
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'X'
            best = min(best, minimax(board, depth + 1, True))
            board[i] = ' '

return best

board = [' '] * 9

print("Best move:", minimax(board, 0, True))

```

Output:

The screenshot shows a Visual Studio Code editor with a Python script for a minimax algorithm. The script is named 'Exp-13' and is located in the 'AI' folder. The script defines a 'minimax' function that takes a board, depth, and is_max as arguments. It implements a minimax algorithm for a 3x3 tic-tac-toe board. The board is represented as a list of 9 elements, where ' ' represents an empty cell, 'O' represents a cell occupied by O, and 'X' represents a cell occupied by X. The function returns the best move for the player whose turn it is to move.

The terminal output shows the execution of the script. The command prompt is '> & C:\Users\adjan\AppData\Local\Programs\Python\Python313\python.exe "C:\Users\adjan\Dropbox\Camera Uploads\SUBISH\college work\Outputs for record\AI\Exp-13"'. The output is 'Best move: 1'.

```

1 def minimax(board, depth, is_max):
2     if 'O' in board:
3         return 1
4     if 'X' in board:
5         return -1
6     if ' ' not in board:
7         return 0
8
9     if is_max:
10        best = -100
11        for i in range(9):
12            if board[i] == ' ':
13                board[i] = 'O'
14                best = max(best, minimax(board, depth + 1, False))
15                board[i] = ' '
16        return best
17    else:
18        best = 100
19        for i in range(9):
20            if board[i] == ' ':
21                board[i] = 'X'
22                best = min(best, minimax(board, depth + 1, True))
23                board[i] = ' '
24
25        return best
26
27 board = [' '] * 9
28
29 print("Best move:", minimax(board, 0, True))

```

Best move: 1

Result

The Minimax Algorithm was successfully implemented in Python to determine the best possible move in a two-player game.

Conclusion

The Minimax Algorithm helps a player make the optimal decision by evaluating possible future moves and selecting the best outcome.